Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

28 July 2025

Topics

- Introduction
- Project management
- Engineering methods
  - Waterfall
  - Agile
  - Scrum
- OOP principles
  - inheritance/encapsulation/polymorphism
  - SOLID
  - DOD data-oriented design
- Software quality
  - metrics
  - testing
  - security
- Design patterns
  - patterns (Gang of Four)
  - antipatterns
- Architecture
- Professional conduct
  - AI & ethics
- Specialty subjects
  - TBD by vote

Textbooks

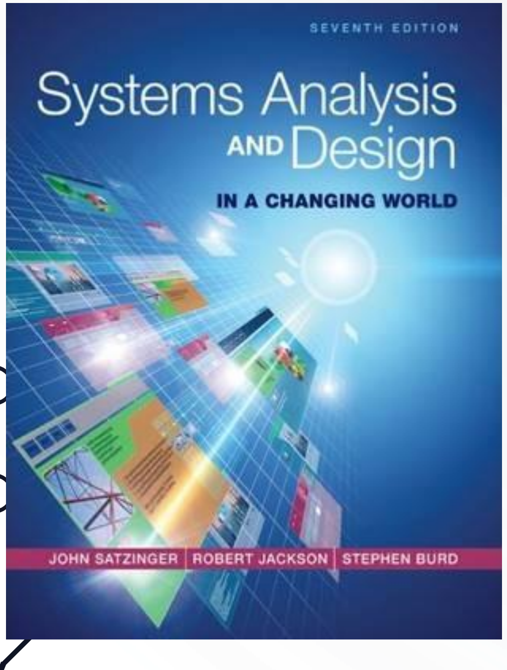
Recommended:

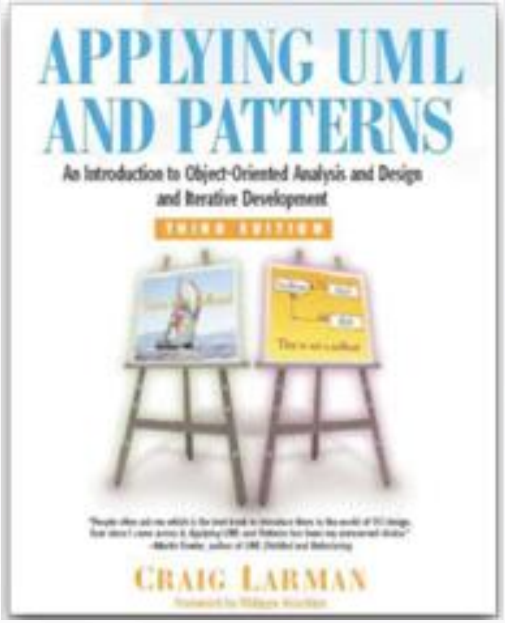
John Satzinger et al, *Introduction to Systems Analysis and Design*

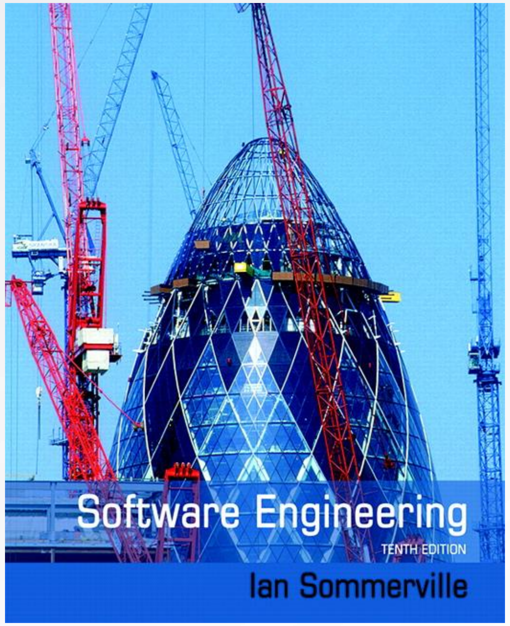
Ian Sommerville, *Software Engineering*

Craig Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*

Erich Gamma et al, *Design Patterns*


SEVENTH EDITION
Systems Analysis AND Design
IN A CHANGING WORLD
JOHN SATZINGER | ROBERT JACKSON | STEPHEN BURD


APPLYING UML AND PATTERNS
An Introduction to Object-Oriented Analysis and Design and Iterative Development
THIRD EDITION
CRAIG LARMAN


Software Engineering
TENTH EDITION
Ian Sommerville

Design Patterns
Elements of Reusable Object-Oriented Software
Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides
Foreword by Grady Booch
ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

Key Activities – Week 1

- **Read** the Capstone Project Overview (available under Content/Capstone Project) https://amathuba.uct.ac.za/d2l/le/lessons/131804/topics/3338958
- **Submit** your project choice by Wednesday https://csc3003s-capstone.cs.uct.ac.za/
- **Take** the Quiz (for marks, TBA)
- **Attend** all the lectures Mon..Thu
- **Read** Sommerville, Chapter 1: Introduction, Chapter 2: Software Processes, and Chapter 8: Software Testing
- **Read** Bennett Chapter 22: Managing Object-Oriented Projects [Amathuba/Content/Week 1][Additional Resources (online)]
- **Attend** your first client meeting Friday

Learning Activities

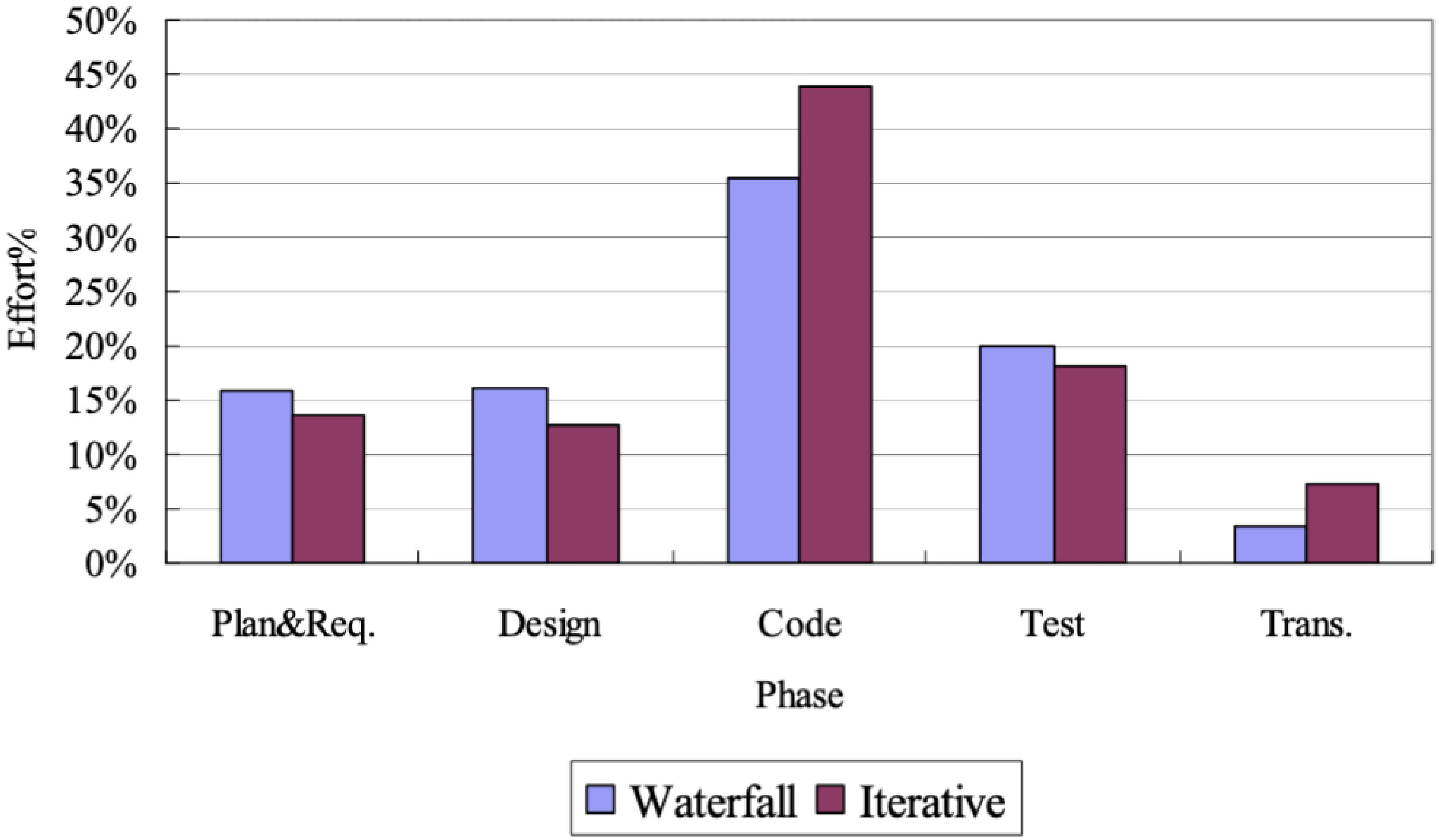
1. Set up your account on the CS Gitlab server (see instructions)
2. Do git tutorial (pick one or two with your team).
3. Do ASD Tutorial from lecture ASD01 to review your UML case studies and sequence diagrams
    - Watch the case study for the tutorial (slides in lecture PPTx)
    - UML Tutorial Part 1 - find a friend or exchange answers within your group for peer marking. [Tutorial] [Answer]
    - UML Tutorial Part 2 - start from the Part 1 solution, find a friend or exchange answers within your group for peer marking [Tutorial] [Answer]
4. Do the network analysis task from lecture ASD03 (note: a network analysis problem is frequently given as a test or exam question). The solution is in this password-protected PDF document. The password is the bottom row of activity [I2: Implement 2] as a six digit number.

What is Software development?

- Requirements gathering
- Software design
- Documentation
- Coding
- Testing
- Bug fixing <= most time goes here
- Maintenance

Large Teams
6+ month
timescales
Maintenance

Where does the time go?



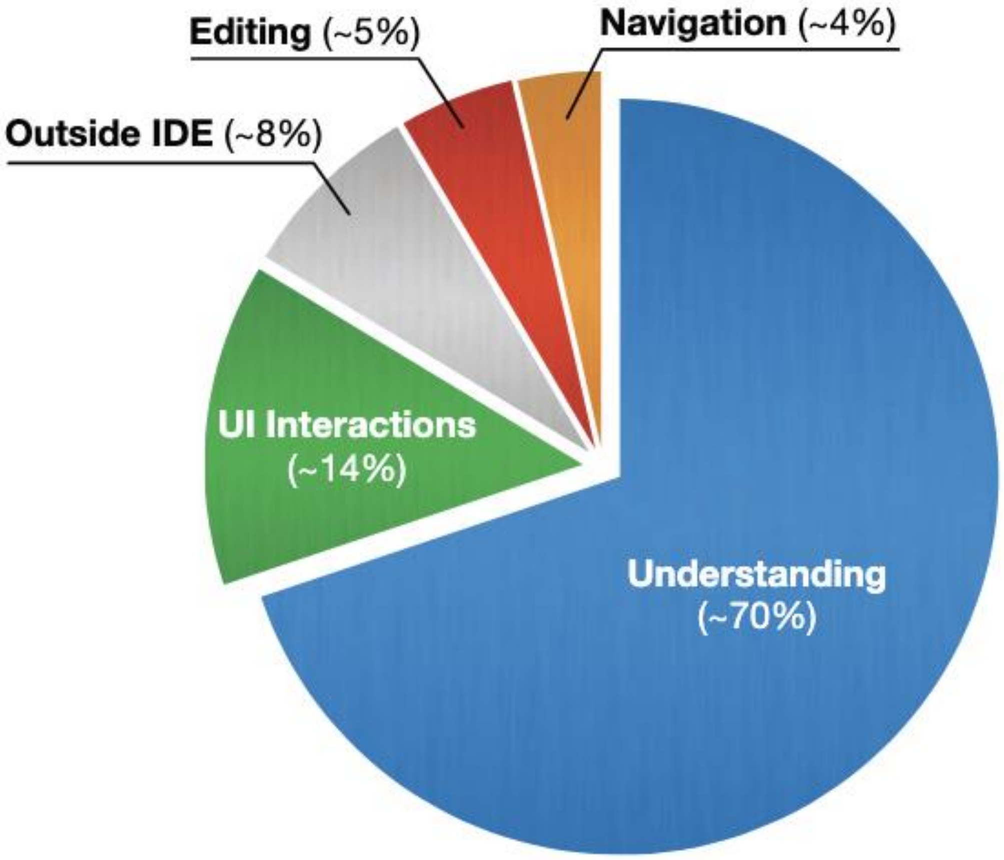


Fig. 3. How do developers spend their time?

R. Minelli et al. *I Know What You Did Last Summer An Investigation of How Developers Spend Their Time*, 2015
Ye, et al. *Phase Distribution of Software Development Effort*, 2008

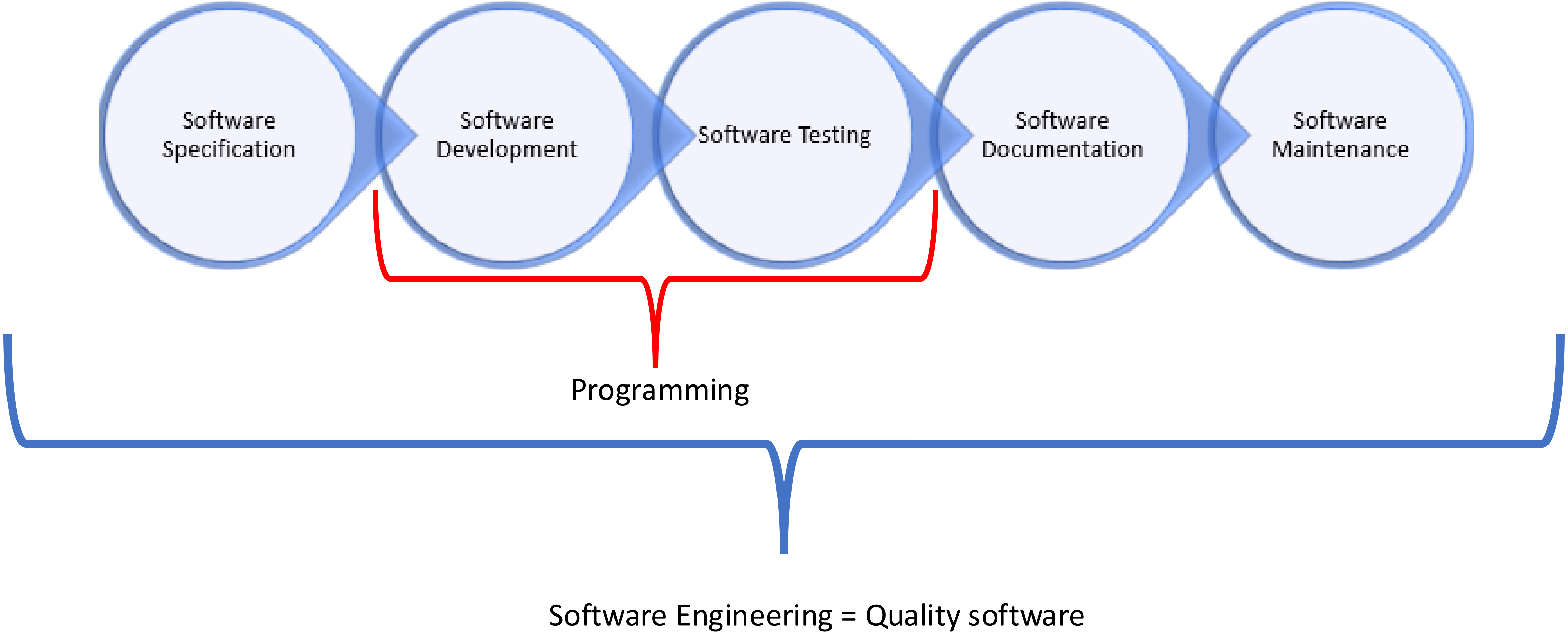
Programmer vs software engineer

Soon to be replaced by ChatGPT?

A programmer is a professional who builds software according to a specification

SOFTWARE ENGINEERS

A Software Engineer is a professional who applies the principles of software engineering for designing, development, maintenance, testing, and evaluation of computer software



Software Specification

Software Development

Software Testing

Software Documentation

Software Maintenance

Programming

Software Engineering = Quality software

History of software engineering

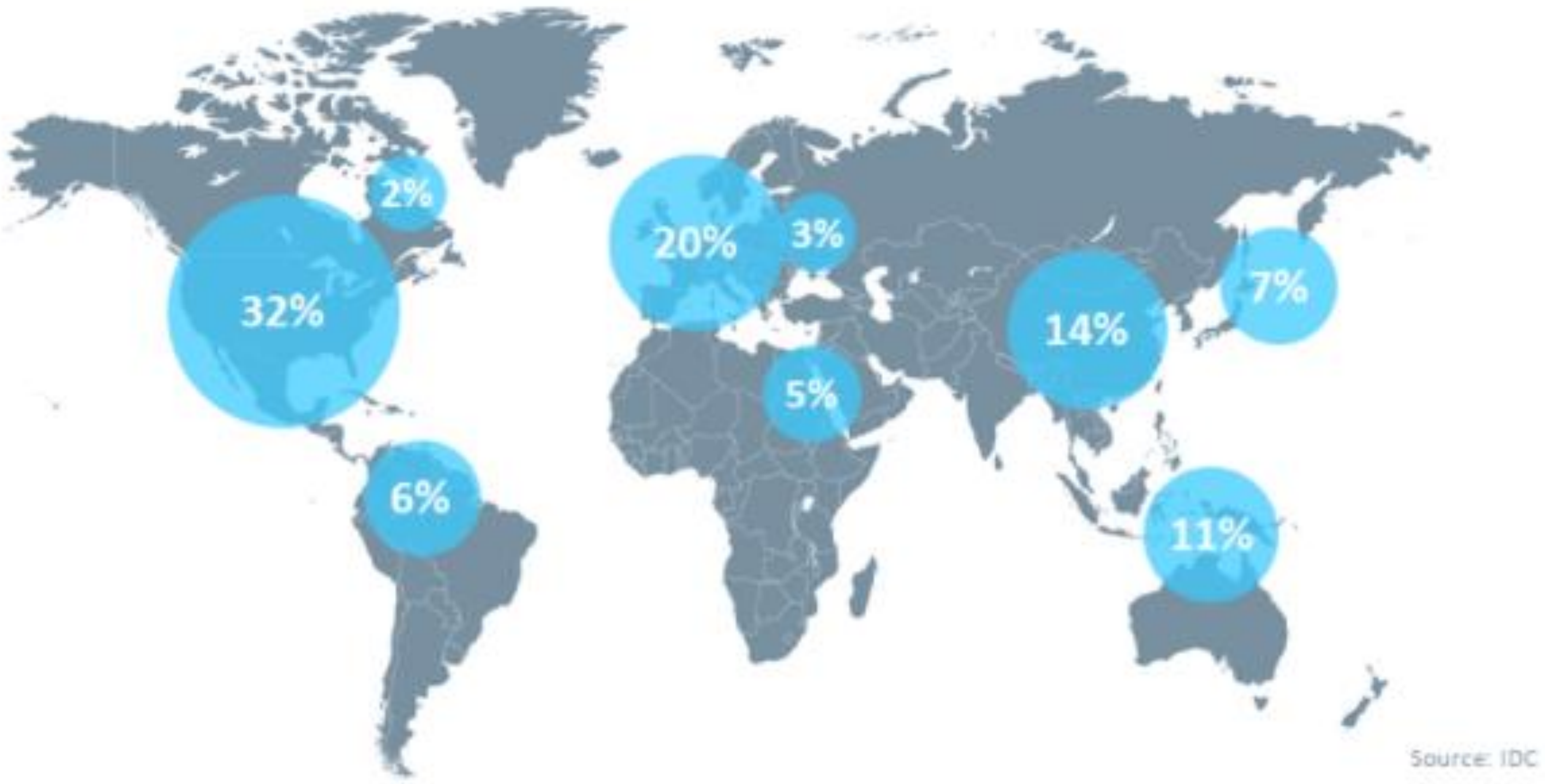
The notion of 'software engineering' was first proposed in 1968 at a conference held to discuss what was then called the 'software crisis' (Naur and Randell, 1969). It became clear that individual approaches to program development did not scale up to large and complex software systems. These were unreliable, cost more than expected, and were delivered late.

Throughout the 1970s and 1980s, a variety of new software engineering techniques and methods were developed, such as structured programming, information hiding and object-oriented development. Tools and standard notations were developed and are now extensively used.

http://www.SoftwareEngineering-9.com/Web/History/

The world relies on software

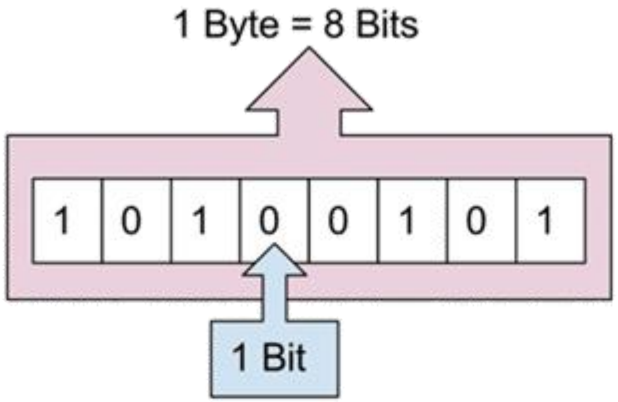
Global spending for 2025 $5.75 trillion


2%
32%
20%
3%
5%
14%
7%
6%
11%

Source: IDC

Yet Software still fails :

- **Increasing demands**
  - As new software engineering techniques help us to build larger, more complex systems, the demands change**.**
  - **Systems must be built and delivered more quickly.**
  - ***Existing software engineering methods cannot cope, and new software engineering techniques have to be developed to meet new these new demands***.
- **Low expectations**
  - It is relatively easy to write computer programs without using software engineering methods and techniques.
  - **Consequently, their software is often more expensive and less reliable than it should be**. We need better software engineering education and training to address this problem



Why does software keep failing?

Better tools, but...

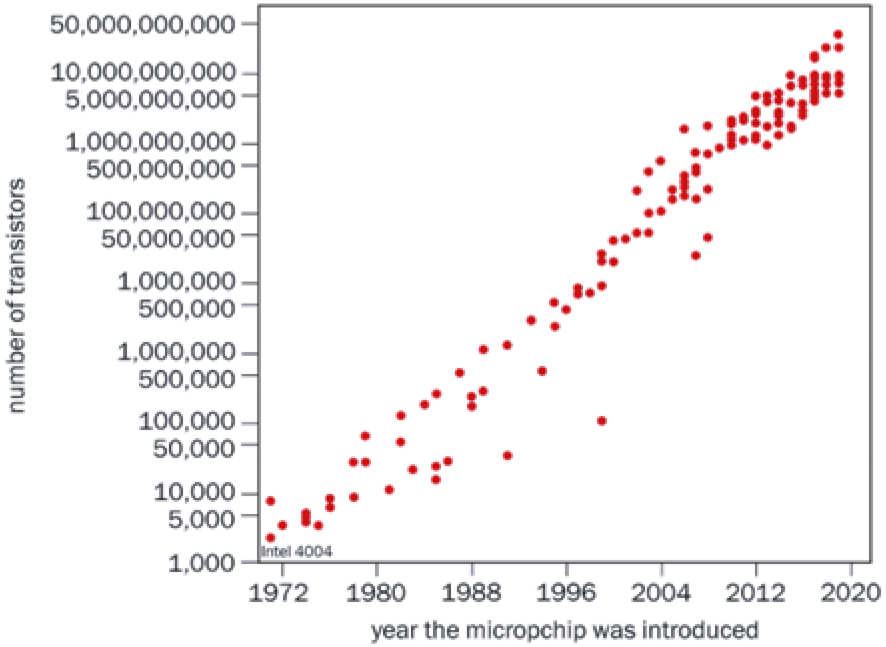
- Complexity grows faster than tools
- ... Human brains

1 byte = 8 bits
1 kilobyte = 1024 bytes
1 megabyte = 1024 kilobyte
1 gigabyte = 1024 megabyte
1 terabyte = 1024 gigabyte

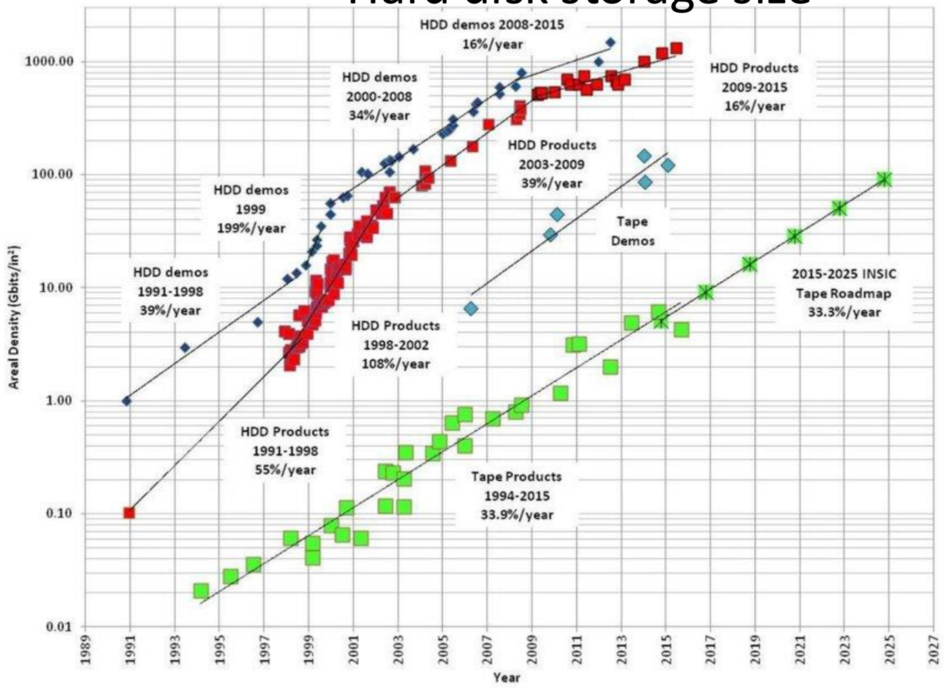
PC program spans 10 orders of magnitude

Networked app spans 13 orders of magnitude

Transistors in CPU



Hard disk storage size



Risk

Risk = cost × likelihood

	cheap ⍰	expensive 💵 💶 💷
unlikely ⚡	Accept 😚	Transfer (insure)
likely 🌧	Reduce	Avoid ☠

Quality aspects of information

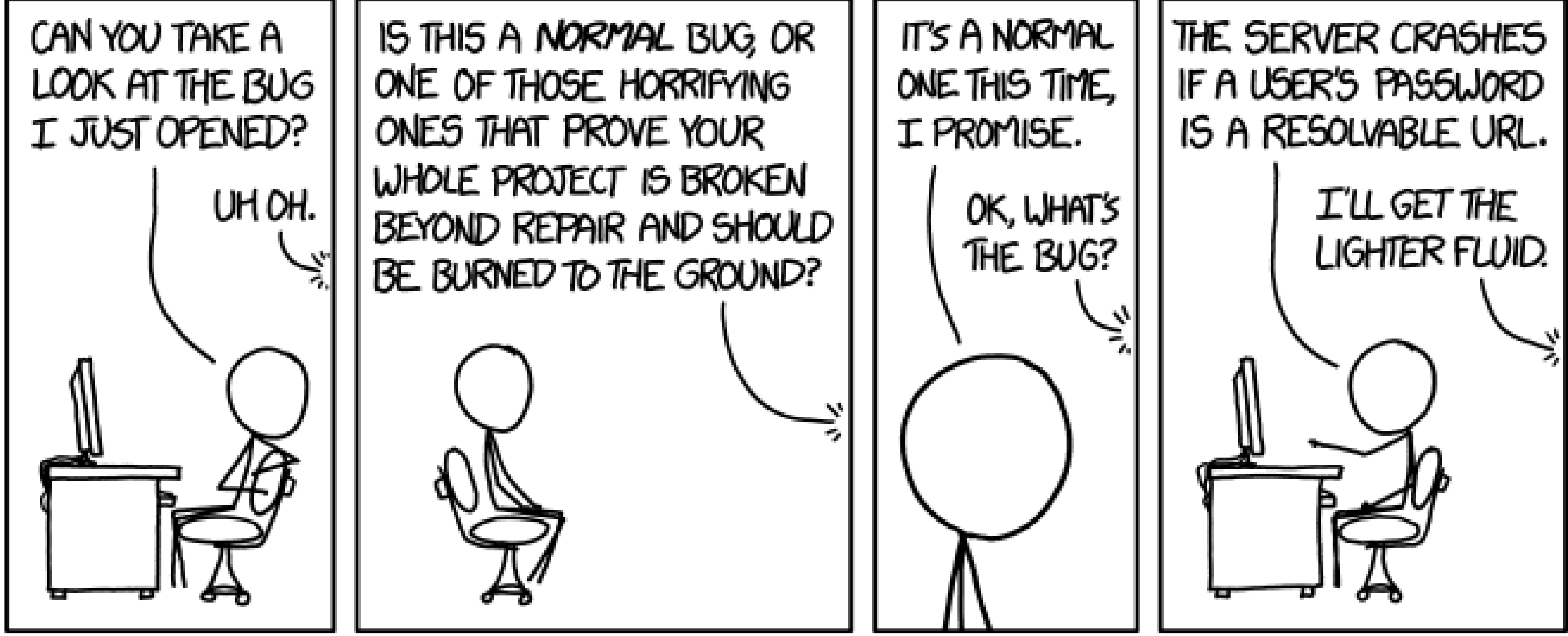
Complete	not too much (*the whole truth*)	Accurate
Correct	not too little (*and nothing but the truth*)	
Timely	correct timestamp (*robbing Peter to pay Paul*)	
mathematical accuracy	correct calculations (*Excel sheets!*)	
Legible	can be read/deciphered (*binary file formats*)	Usable
Relevant	fits info demand/not outdated	
Understandable	incl. assumptions	
Comparable	both in time and against other data	
Verified	has it been/can it be checked?	
Trustworthy	believable (source?)	
Legally compliant	violates no laws (*e.g.: VAT calculations*)	Secure
Ethical	privacy/medical	
Authorized	user allowed to access?	

ISO 25010 – 2011 Software quality metrics

Software quality								
Suitable	**Efficient**	**Compatible**	**Usable**	**Reliable**	**Secure**	**Maintainable**	**Flexible**	**Safe**
Complete	Time	Co-existence	recognizable	fault-free	confidential	modular	adaptable	operations
Correct	Resources	interoperable	learnable	availability	integrity	reusable	scalable	id risks
Appropriate	Capacity		user error	fault tolerant	non-repudiation	analyzable	installable	fail-safe
			engagement	recoverability	accountability	modifiable	replaceable	hazard warn
			inclusivity		authenticity	testable		safe integration
			assistance		resistance			
			self-describe					

Heitlager et al: A Practical Model for Measuring Maintainability
https://iso25000.com/index.php/en/iso-25000-standards/iso-25010/

Functionality


CAN YOU TAKE A LOOK AT THE BUG I JUST OPENED?
UH OH.
IS THIS A NORMAL BUG, OR ONE OF THOSE HORRIFYING ONES THAT PROVE YOUR WHOLE PROJECT IS BROKEN BEYOND REPAIR AND SHOULD BE BURNED TO THE GROUND?
IT'S A NORMAL ONE THIS TIME, I PROMISE.
OK, WHAT'S THE BUG?
THE SERVER CRASHES IF A USER'S PASSWORD IS A RESOLVABLE URL.
I'LL GET THE LIGHTER FLUID.

• Suitability	(meets requirements)
• Accuracy	(13 aspects of information quality)
• Interoperable	(works with outside info/systems [input + output])
• Security	Needs to stand on its own

Reliability and security

The code
Security of code
Security in code

Reliable software:

- is mature (incl. frameworks)
- doesn’t crash
- stays within storage limits
- is redundant
- is fault-tolerant
- runs on future hardware

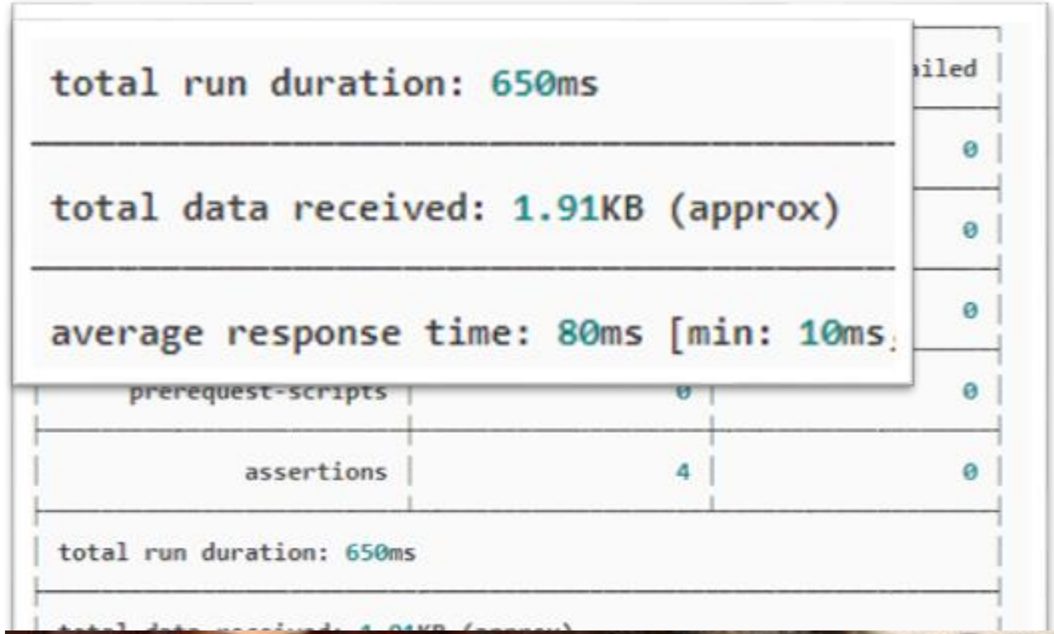
Secure software:

- does not accept faulty input
- cannot be hacked
- does not leak data
- cannot be used beyond its intended purpose
- complies with privacy/ethical/legal req.

Reliability & security must be designed in from the start

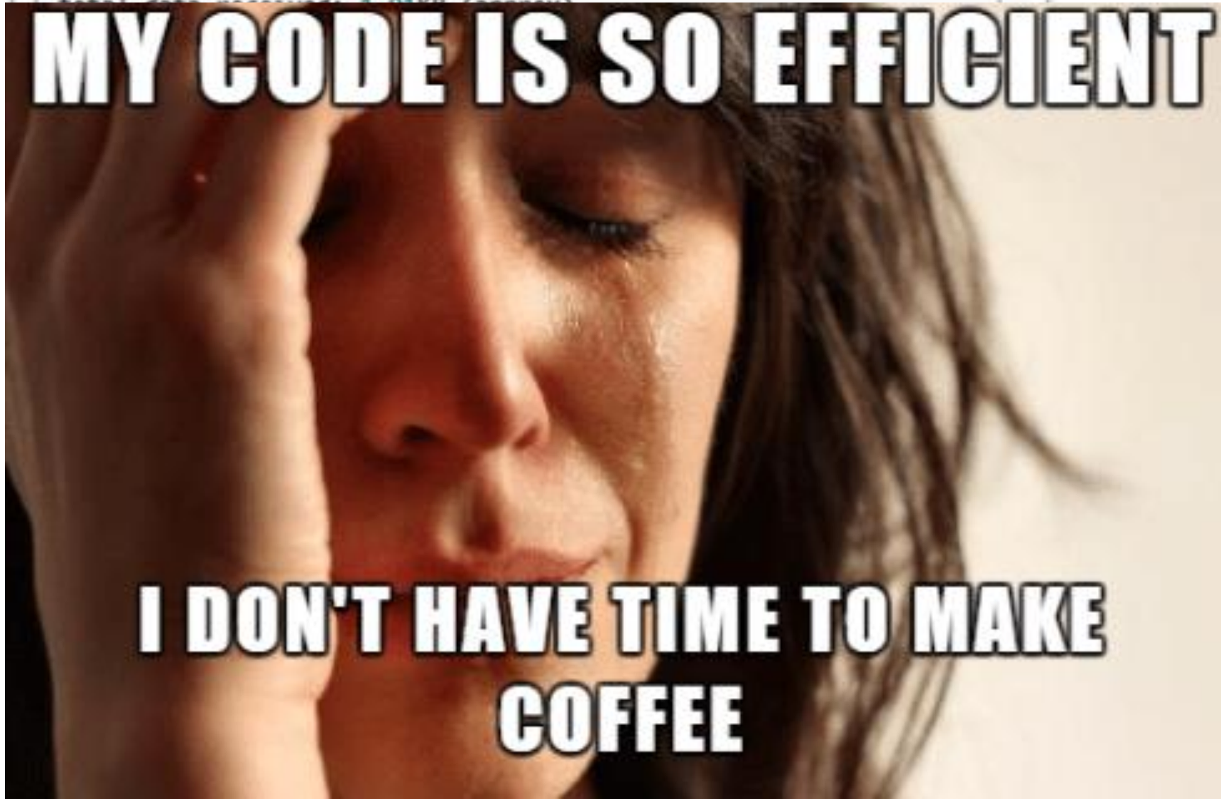
Efficiency

- Run time
  (time to completion)
- Quality of solution
  (solvers)
- Latency
  (interactive software)
- Resource use
  (storage/network/memory)
- Power
  (electricity use)

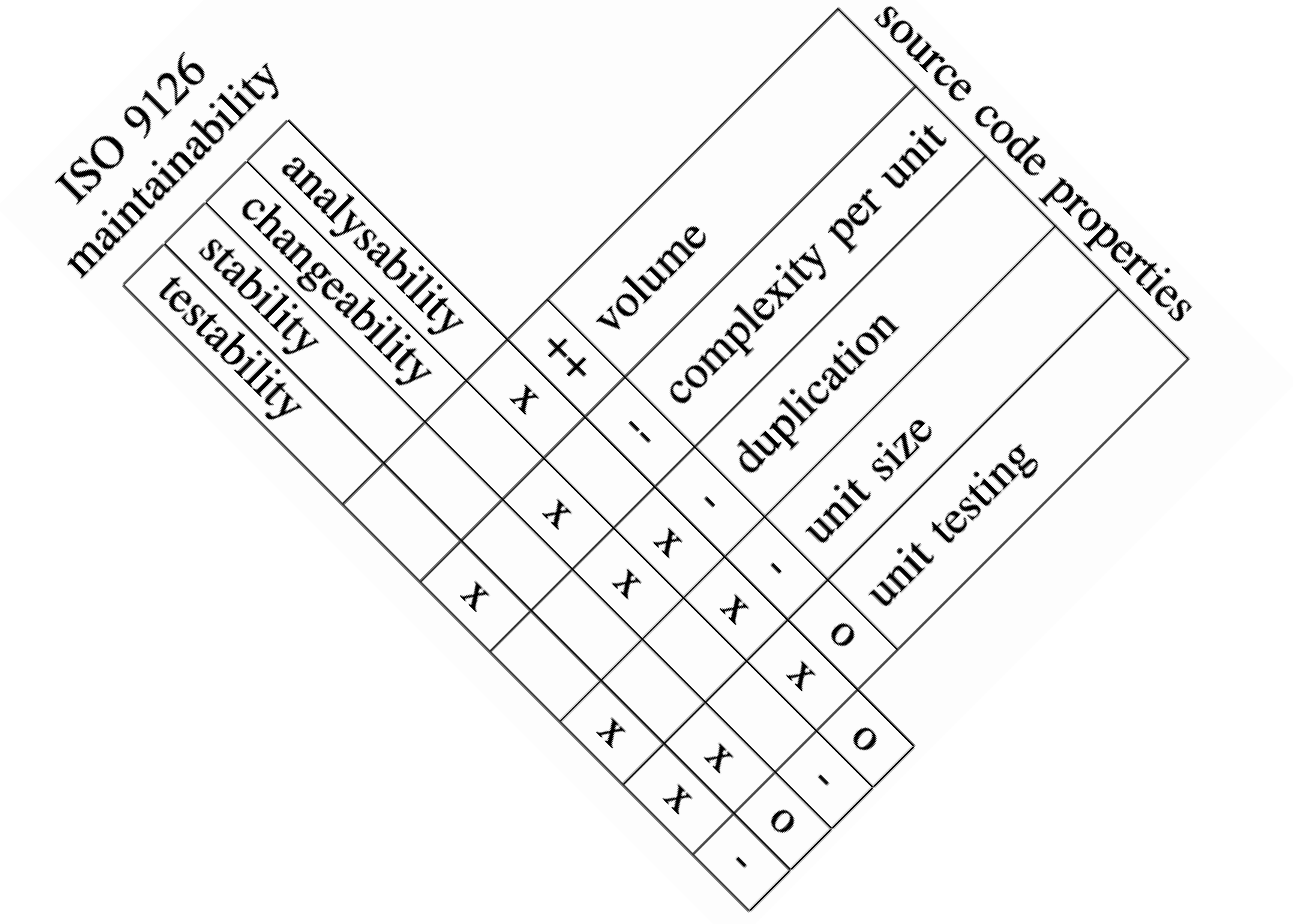


```
total run duration: 650ms

total data received: 1.91KB (approx)

average response time: 80ms [min: 10ms
```

```
                                      iled
                                         0
                                         0
                                         0
   prerequest-scripts |      0 |         0
           assertions |      4 |         0
total run duration: 650ms
```


MY CODE IS SO EFFICIENT
I DON'T HAVE TIME TO MAKE COFFEE

Maintainability: ISO 9126



ISO 9126 maintainability	source code properties					
	volume	complexity per unit	duplication	unit size	unit testing	
	++	--	-	-	o	
analysability	x		x	x	x	o
changeability		x	x			-
stability					x	o
testability		x		x	x	-

duplication	rank
0-3%	++
3-5%	+
5-10%	0
10-20%	–
20-100%	––

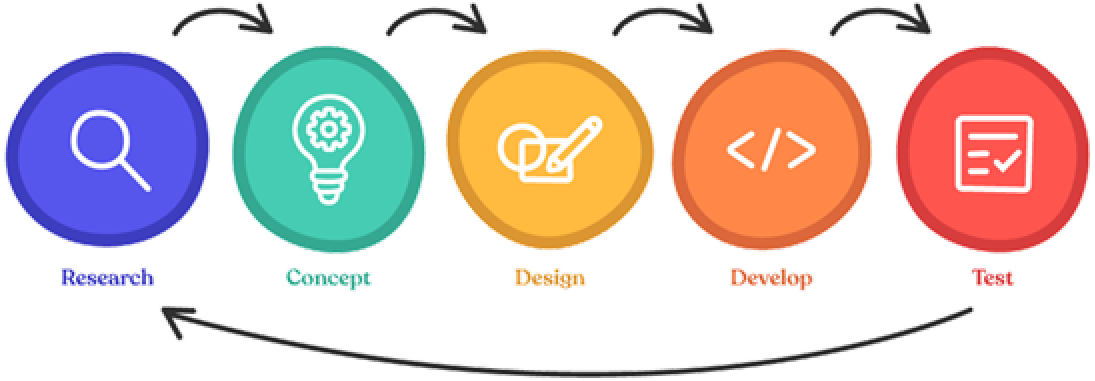
test coverage	rank
95-100%	++
80-95%	+
60-80%	0
20-60%	–
0-20%	––

Usability:
ISO 9241 *Ergonomics of Human System Interaction*

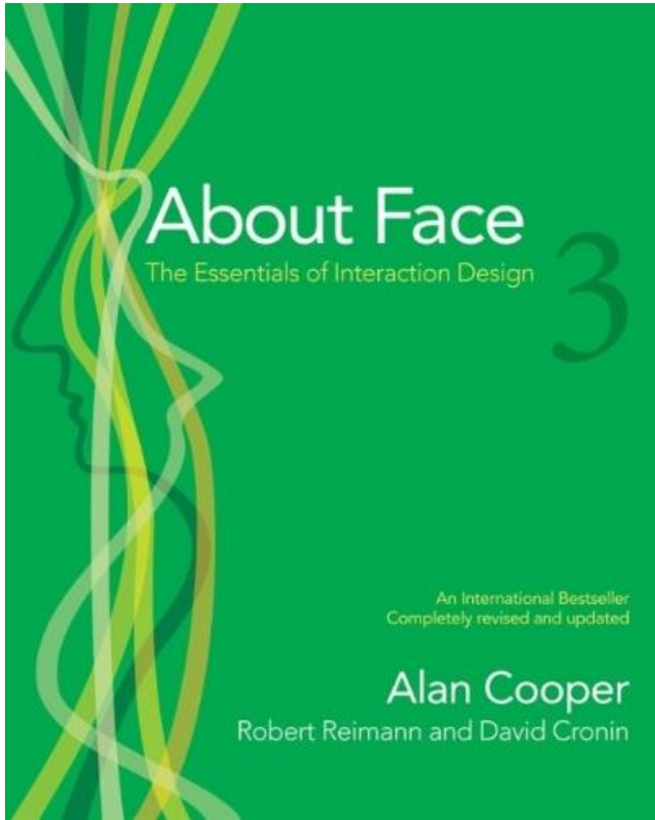
UCT CS:

University of Cape Town • iYunivesithi yaseKapa • Universiteit van Kaapstad

User centered design

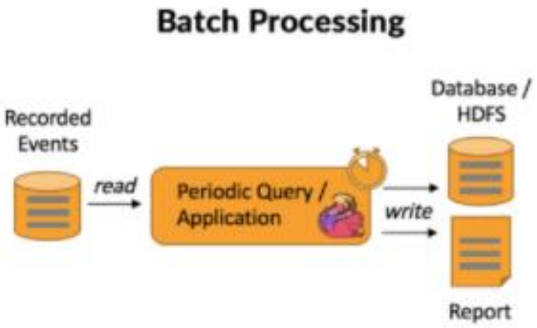

Research
Concept
Design
Develop
Test

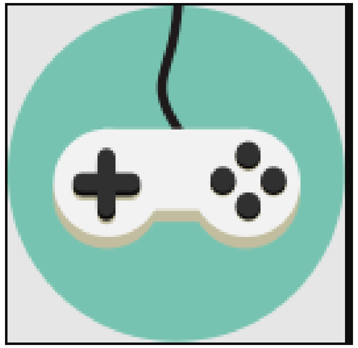
- Learnability
  - Ease of use
    - Mice + elephants
- Understandability
- Operability
  - Usable
  - Useful
  - Error-free
- Attractiveness
  - Fast
  - Pretty


About Face
The Essentials of Interaction Design
3
An International Bestseller
Completely revised and updated
Alan Cooper
Robert Reimann and David Cronin

Process

Embedded Systems


Batch Processing
Recorded Events
read
Periodic Query / Application
write
Database / HDFS
Report

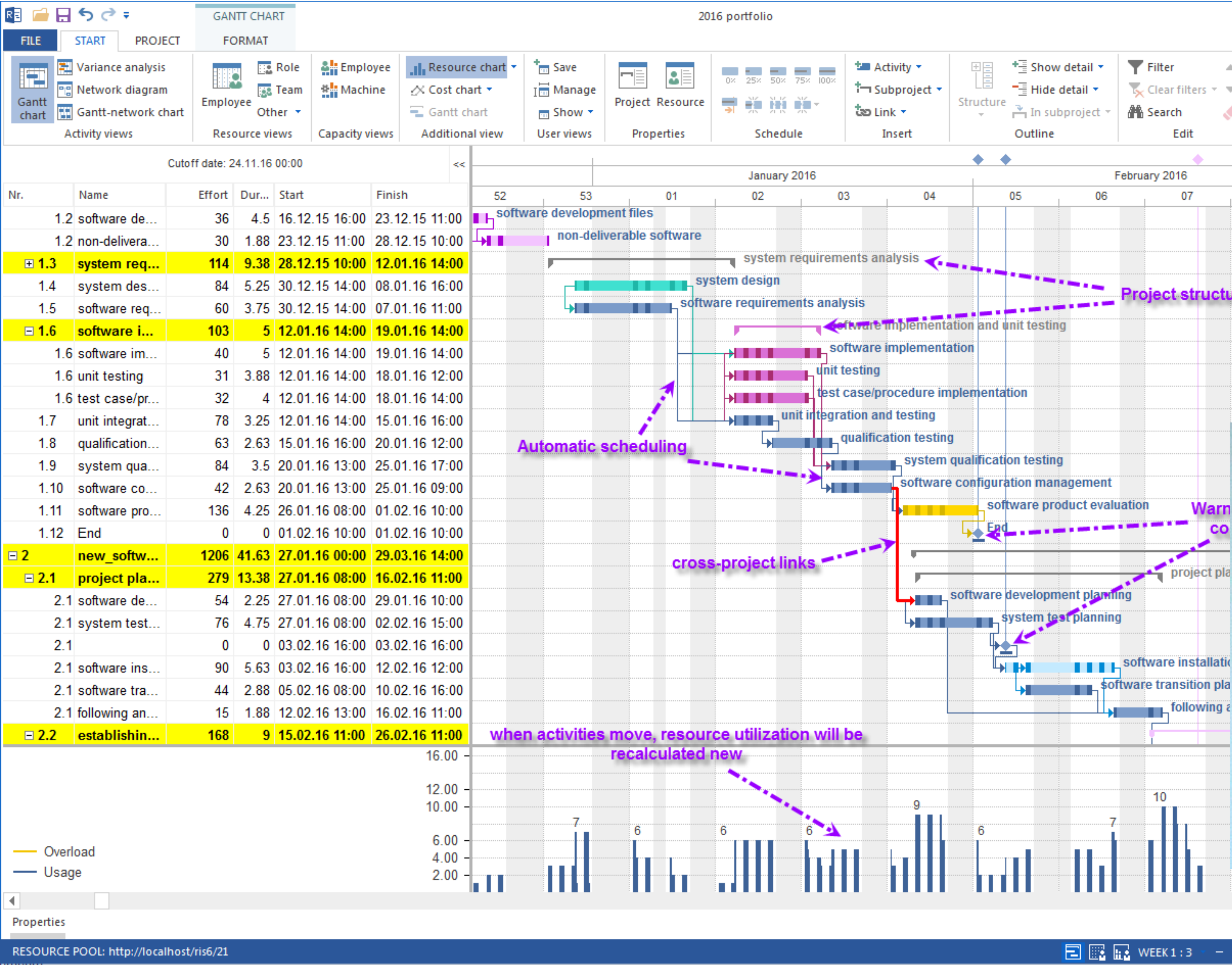


You use different software engineering techniques for each type of system because the software has quite different characteristics.

However – The following is true for any kind of software system

- Managed and understood development process
- Focus on dependability and performance
- Understand and manage software requirements
- Effective use/ reuse of resources

Planning


FILE
START
PROJECT
GANTT CHART
FORMAT
Gantt chart
Variance analysis
Network diagram
Gantt-network chart
Activity views
Employee
Role
Team
Other
Resource views
Employee
Machine
Capacity views
Resource chart
Cost chart
Gantt chart
Additional view
Save
Manage
Show
User views
Project
Resource
Properties
Schedule
Activity
Subproject
Link
Insert
Structure
Show detail
Hide detail
In subproject
Outline
Filter
Clear filters
Search
Edit
Cutoff date
Current date
Project start
Scrolling
January 2016
February 2016
52
53
01
02
03
04
05
06
07
08
software development files
non-deliverable software
system requirements analysis
system design
software requirements analysis
software implementation and unit testing
software implementation
unit testing
test case/procedure implementation
unit integration and testing
qualification testing
system qualification testing
software configuration management
software product evaluation
End
project pla
software development planning
system test planning
software installatio
software transition pla
following a
16.00
12.00
10.00
6.00
4.00
2.00
7
6
6
6
9
6
7
10
Overload
Usage
Properties
WEEK 1 : 3

2016 portfolio

Cutoff date: 24.11.16 00:00

Nr.	Name	Effort	Dur...	Start	Finish
1.2	software de...	36	4.5	16.12.15 16:00	23.12.15 11:00
1.2	non-delivera...	30	1.88	23.12.15 11:00	28.12.15 10:00
⊞ 1.3	**system req...**	**114**	**9.38**	**28.12.15 10:00**	**12.01.16 14:00**
1.4	system des...	84	5.25	30.12.15 14:00	08.01.16 16:00
1.5	software req...	60	3.75	30.12.15 14:00	07.01.16 11:00
⊟ 1.6	**software i...**	**103**	**5**	**12.01.16 14:00**	**19.01.16 14:00**
1.6	software im...	40	5	12.01.16 14:00	19.01.16 14:00
1.6	unit testing	31	3.88	12.01.16 14:00	18.01.16 12:00
1.6	test case/pr...	32	4	12.01.16 14:00	18.01.16 14:00
1.7	unit integrat...	78	3.25	12.01.16 14:00	15.01.16 16:00
1.8	qualification...	63	2.63	15.01.16 16:00	20.01.16 12:00
1.9	system qua...	84	3.5	20.01.16 13:00	25.01.16 17:00
1.10	software co...	42	2.63	20.01.16 13:00	25.01.16 09:00
1.11	software pro...	136	4.25	26.01.16 08:00	01.02.16 10:00
1.12	End	0	0	01.02.16 10:00	01.02.16 10:00
⊟ 2	**new_softw...**	**1206**	**41.63**	**27.01.16 00:00**	**29.03.16 14:00**
⊟ 2.1	**project pla...**	**279**	**13.38**	**27.01.16 08:00**	**16.02.16 11:00**
2.1	software de...	54	2.25	27.01.16 08:00	29.01.16 10:00
2.1	system test...	76	4.75	27.01.16 08:00	02.02.16 15:00
2.1		0	0	03.02.16 16:00	03.02.16 16:00
2.1	software ins...	90	5.63	03.02.16 16:00	12.02.16 12:00
2.1	software tra...	44	2.88	05.02.16 08:00	10.02.16 16:00
2.1	following an...	15	1.88	12.02.16 13:00	16.02.16 11:00
⊟ 2.2	**establishin...**	**168**	**9**	**15.02.16 11:00**	**26.02.16 11:00**

RESOURCE POOL: http://localhost/ris6/21

Project structuring

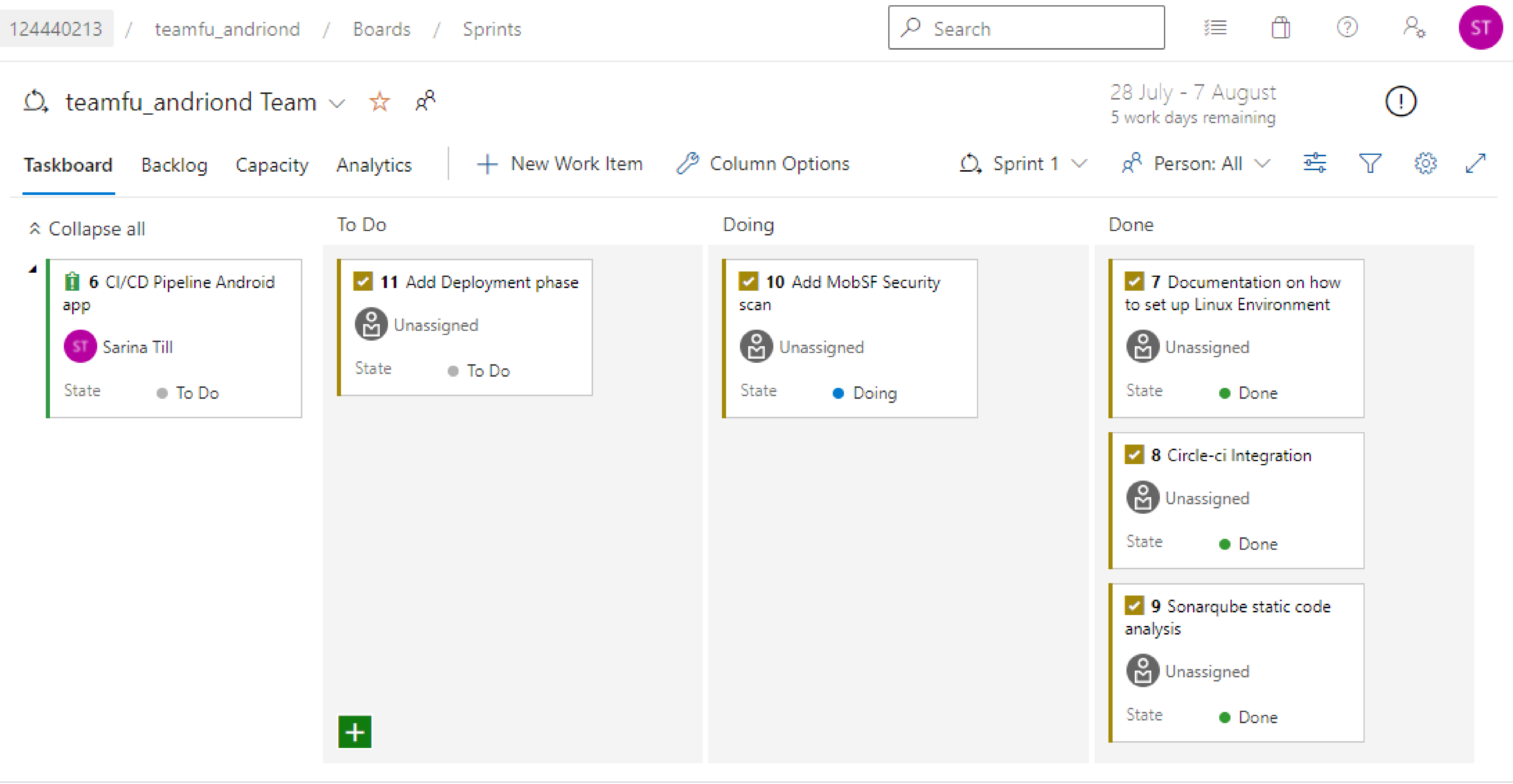
Automatic scheduling

cross-project links

Warn
co

when activities move, resource utilization will be recalculated new

To Do
Doing
Done
USE KANBAN
TRY KANBAN TOOL
LEARN ABOUT KANBAN
GET SOME STICKY NOTES!
GET A WHITE-BOARD


124440213 / teamfu_andriond / Boards / Sprints
Search
ST
teamfu_andriond Team
28 July - 7 August
5 work days remaining
Taskboard
Backlog
Capacity
Analytics
New Work Item
Column Options
Sprint 1
Person: All
Collapse all
To Do
Doing
Done

6 CI/CD Pipeline Android app
Sarina Till
State ● To Do

11 Add Deployment phase
Unassigned
State ● To Do

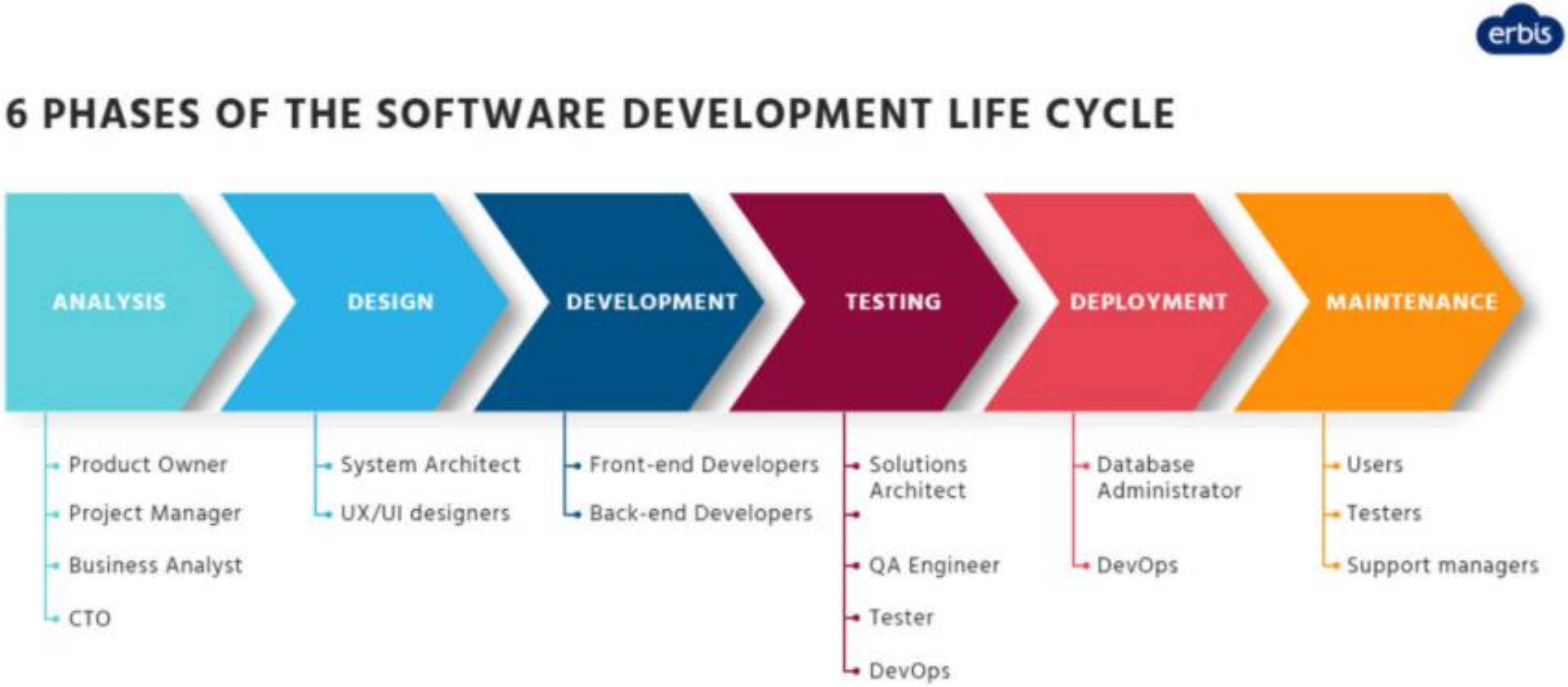
10 Add MobSF Security scan
Unassigned
State ● Doing

7 Documentation on how to set up Linux Environment
Unassigned
State ● Done

8 Circle-ci Integration
Unassigned
State ● Done

9 Sonarqube static code analysis
Unassigned
State ● Done

SDLC: Software development life cycle



6 PHASES OF THE SOFTWARE DEVELOPMENT LIFE CYCLE

ANALYSIS
- Product Owner
- Project Manager
- Business Analyst
- CTO

DESIGN
- System Architect
- UX/UI designers

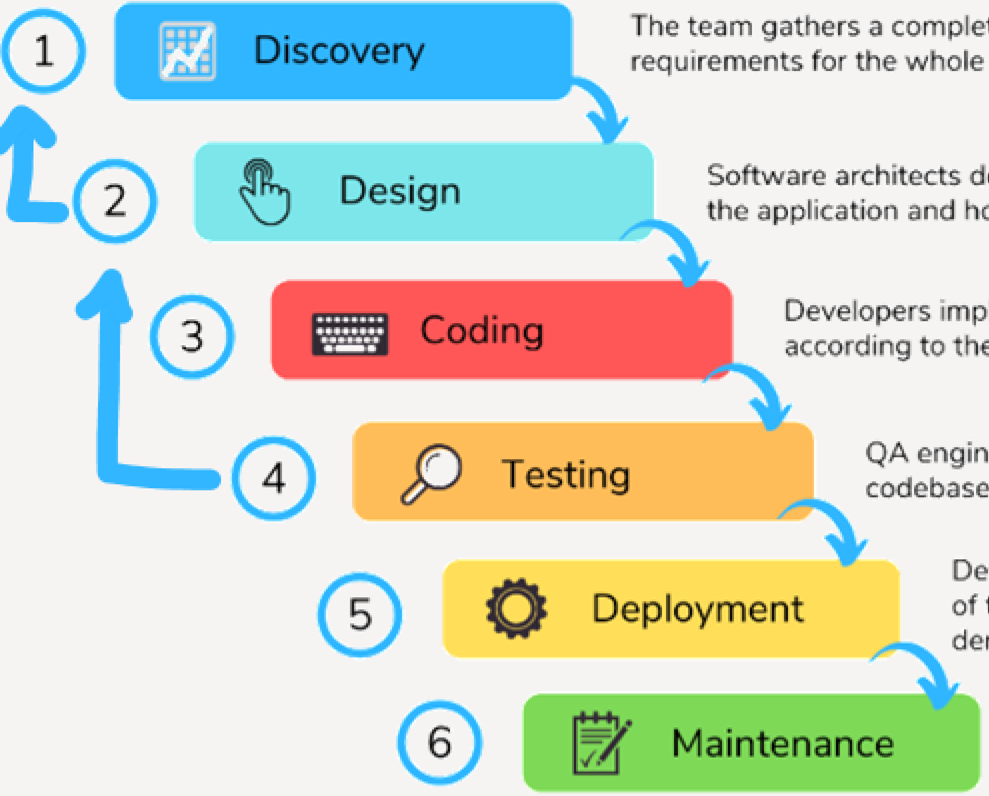
DEVELOPMENT
- Front-end Developers
- Back-end Developers

TESTING
- Solutions Architect
- QA Engineer
- Tester
- DevOps

DEPLOYMENT
- Database Administrator
- DevOps

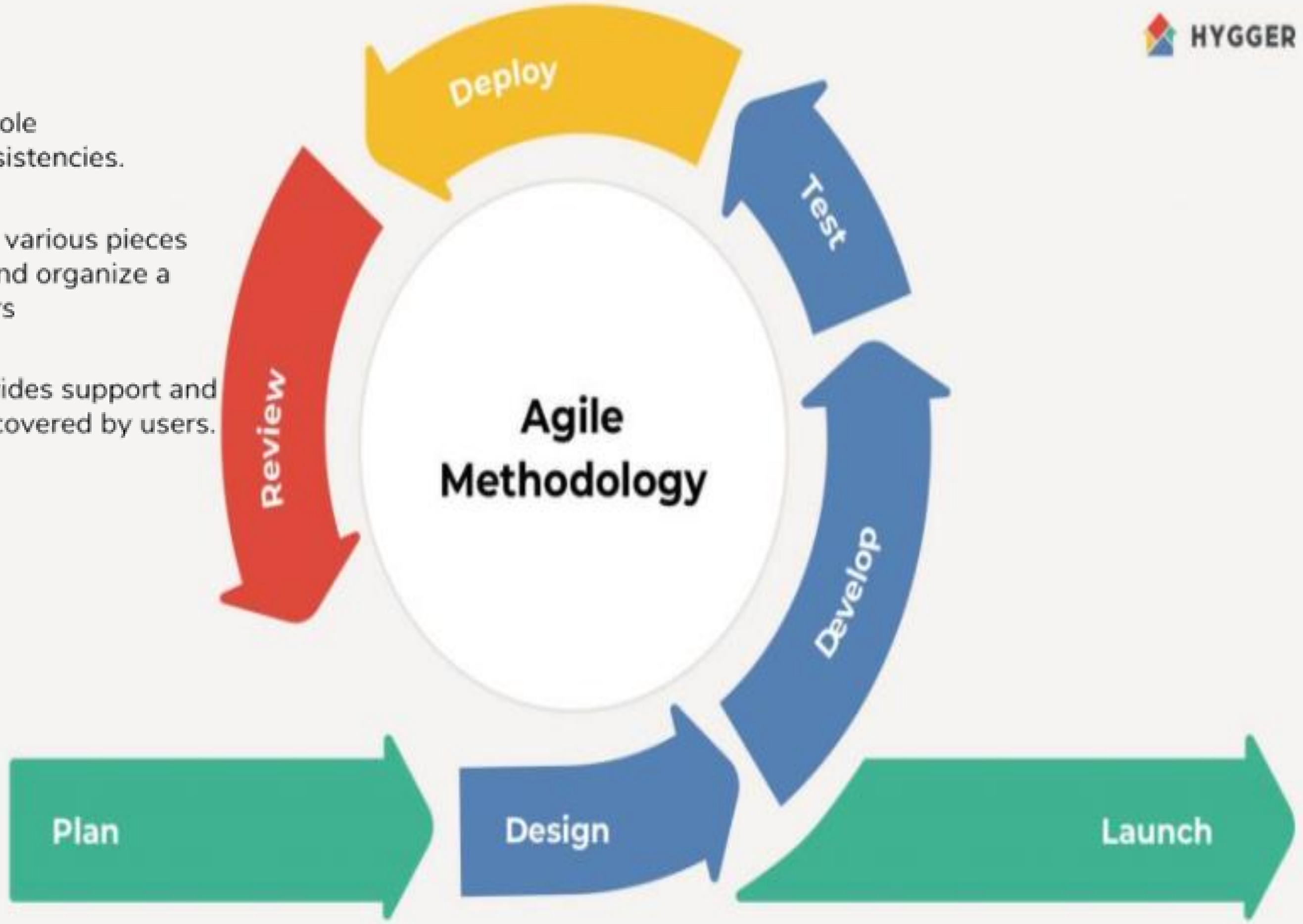
MAINTENANCE
- Users
- Testers
- Support managers

Waterfall model


1
Discovery
2
Design
3
Coding
4
Testing
5
Deployment
6
Maintenance

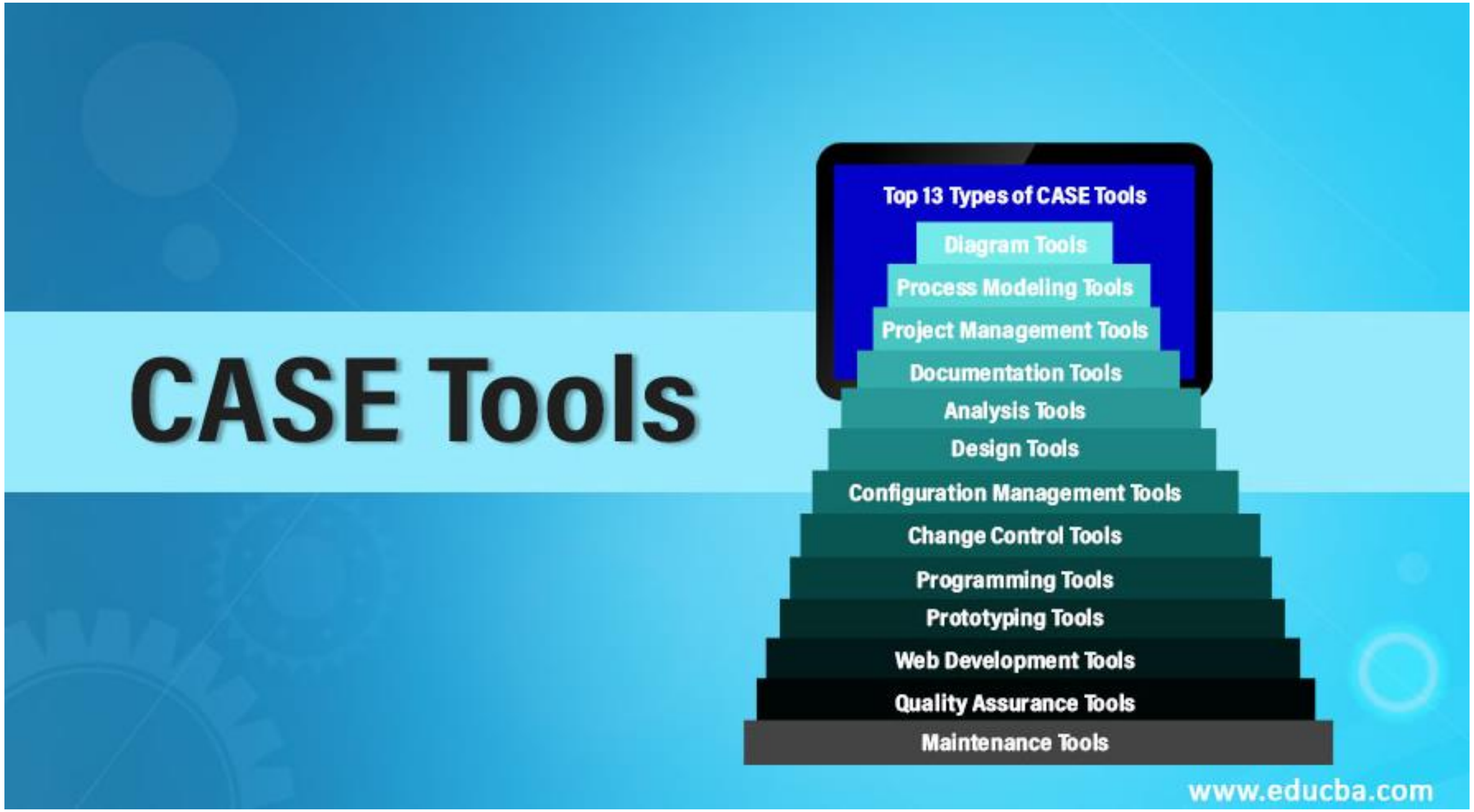
1. Discovery: The team gathers a complete list of requirements for the whole project.
2. Design: Software architects decide how to build the application and how it's going to work.
3. Coding: Developers implement the design according to the requirements.
4. Testing: QA engineers check the whole codebase for bugs or inconsistencies.
5. Deployment: Developers integrate various pieces of the final product and organize a demo for stakeholders
6. Maintenance: The team provides support and fixes bugs discovered by users.

multiple
times


HYGGER
Agile Methodology
Plan
Design
Develop
Test
Deploy
Review
Launch

Tools

- StarUML 👍
- Visio
- LucidChart
- Invision
- Trello
- Monday.com
- Asana
- Slack


CASE Tools
Top 13 Types of CASE Tools
Diagram Tools
Process Modeling Tools
Project Management Tools
Documentation Tools
Analysis Tools
Design Tools
Configuration Management Tools
Change Control Tools
Programming Tools
Prototyping Tools
Web Development Tools
Quality Assurance Tools
Maintenance Tools
www.educba.com

Generative AI

Not allowed for design phase

Now: not allowed for coding phase

Next week: maybe...

- Department is currently working on AI policy
  - Will keep you posted
  - Need student input

- Will have AI lecture as soon as policy clear

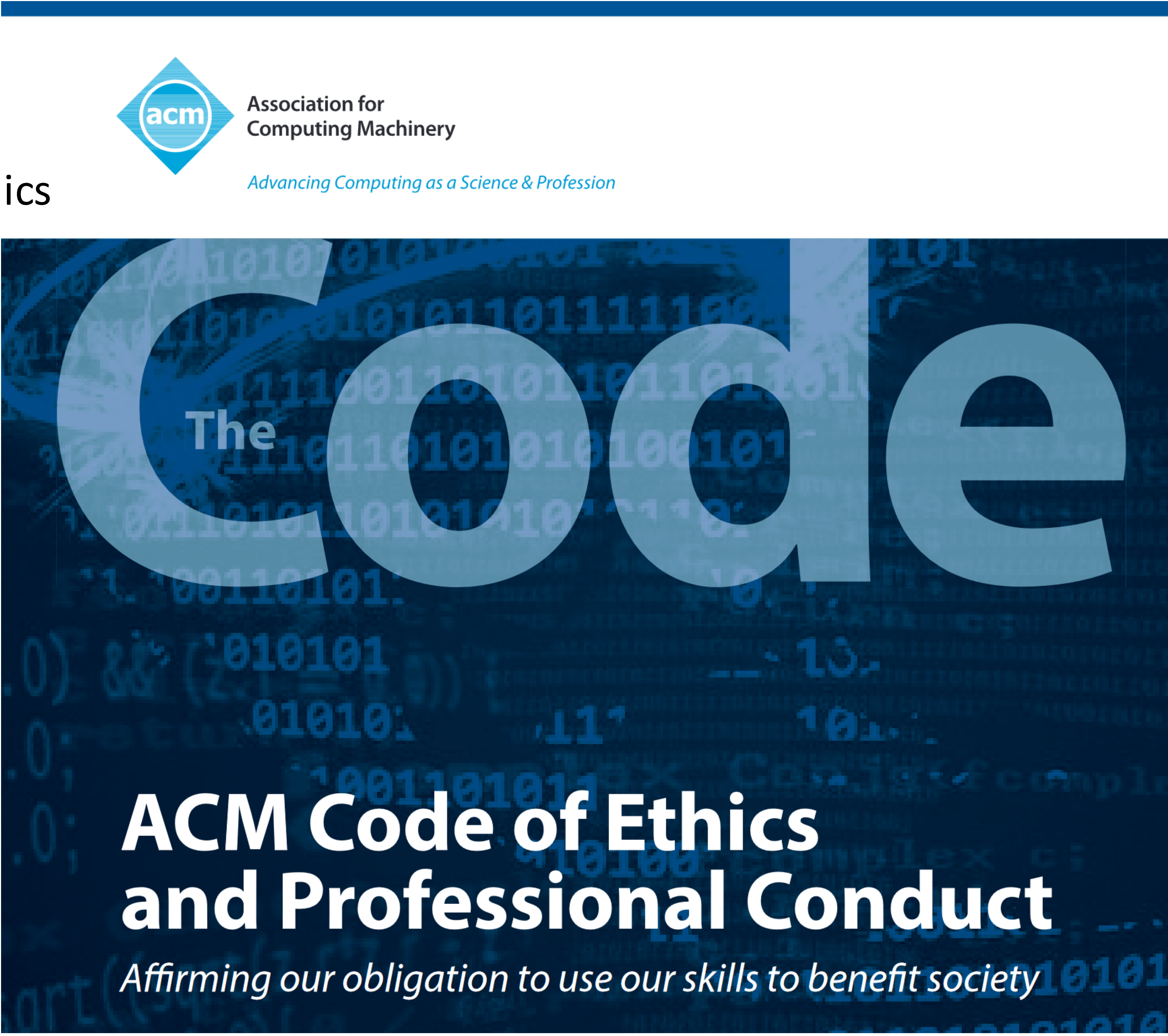
Tests & exam

- Amathuba creates AI generated summary of lectures
  - (Almost) nothing in those summaries will be in the exam

- To pass, will have to
  - Attend lectures
  - Watch lecture recordings
  - Hint: pay close attention to stuff on the blackboard
  - Reading slides good idea, but exam mostly tests understanding, not memorization

Ethics

https://www.acm.org/code-of-ethics



Association for Computing Machinery

Advancing Computing as a Science & Profession

The Code

ACM Code of Ethics and Professional Conduct

Affirming our obligation to use our skills to benefit society

ACM code: General ethical principles.

1.1 Contribute to society and to human well-being, acknowledging that all people are stakeholders in computing.

1.2 Avoid harm.

1.3 Be honest and trustworthy.

1.4 Be fair and do not discriminate.

1.5 Respect IP: the work required to produce new ideas, inventions, creative works, and computing artifacts.

1.6 Respect privacy.

1.7 Respect confidentiality.

ACM code: Professional responsibilities

2.1 Strive to achieve high quality in processes and products

2.2 Maintain high standards of professional competence, conduct, and ethical practice.

2.3 Know and respect existing rules

2.4 Accept and provide appropriate review.

2.5 Give thorough evaluations of computer systems and their impacts, including analysis of possible risks.

2.6 Work only in areas of competence.

2.7 Foster understanding of computing and related technologies, and their consequences.

2.8 Access computing and communication resources only when authorized or when compelled by the public good.

2.9 Design and implement systems that are secure.

ACM code: Professional leadership principles

3.1 Ensure that the public good is the central concern

3.2 promote fulfillment of social responsibilities by members of your organization.

3.3 Manage personnel and resources to enhance the quality of working life.

3.4 Support policies and processes that reflect the principles of the Code.

3.5 Help members of the organization to grow as professionals.

3.6 Use care when modifying or retiring systems.

3.7 Care for vital systems

ACM code: Compliance with the code

4.1 Uphold, promote, and respect the principles of the Code.

4.2 Treat violations of the Code as inconsistent with membership in the ACM.

Source Control – managing a codebase with multiple contributors

Git

- Track changes
- Log of who did what, when
- Mechanism to roll back to previous version
- Mechanism to try different experimental versions of program
  - branches
- Issue tracking
- Way to release code to the world

`git` – the “stupid content tracker”

BitBucket

bitbucket.org

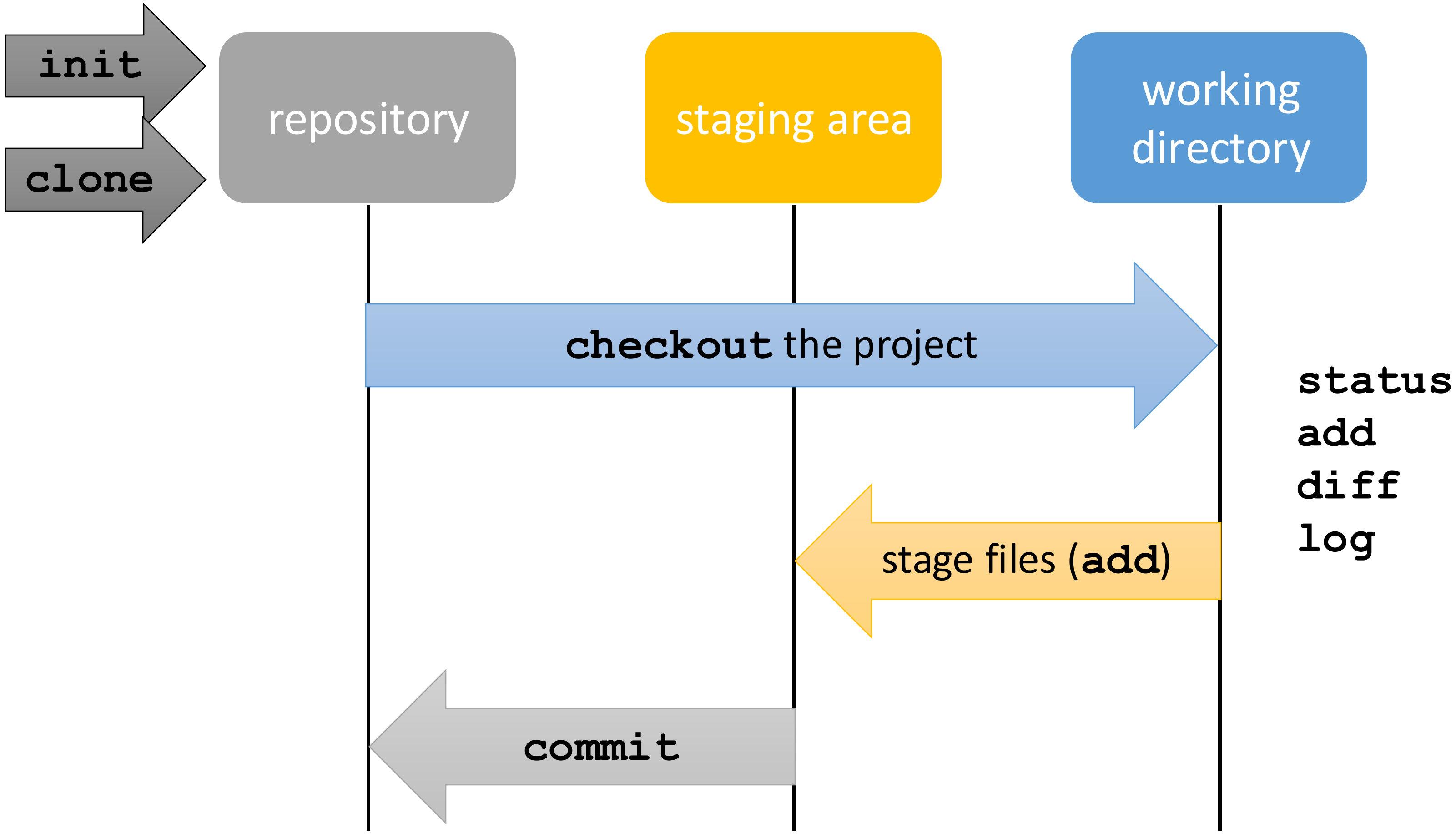
GitHub

github.org

GitLab

gitlab.cs.uct.ac.za

Local Operations

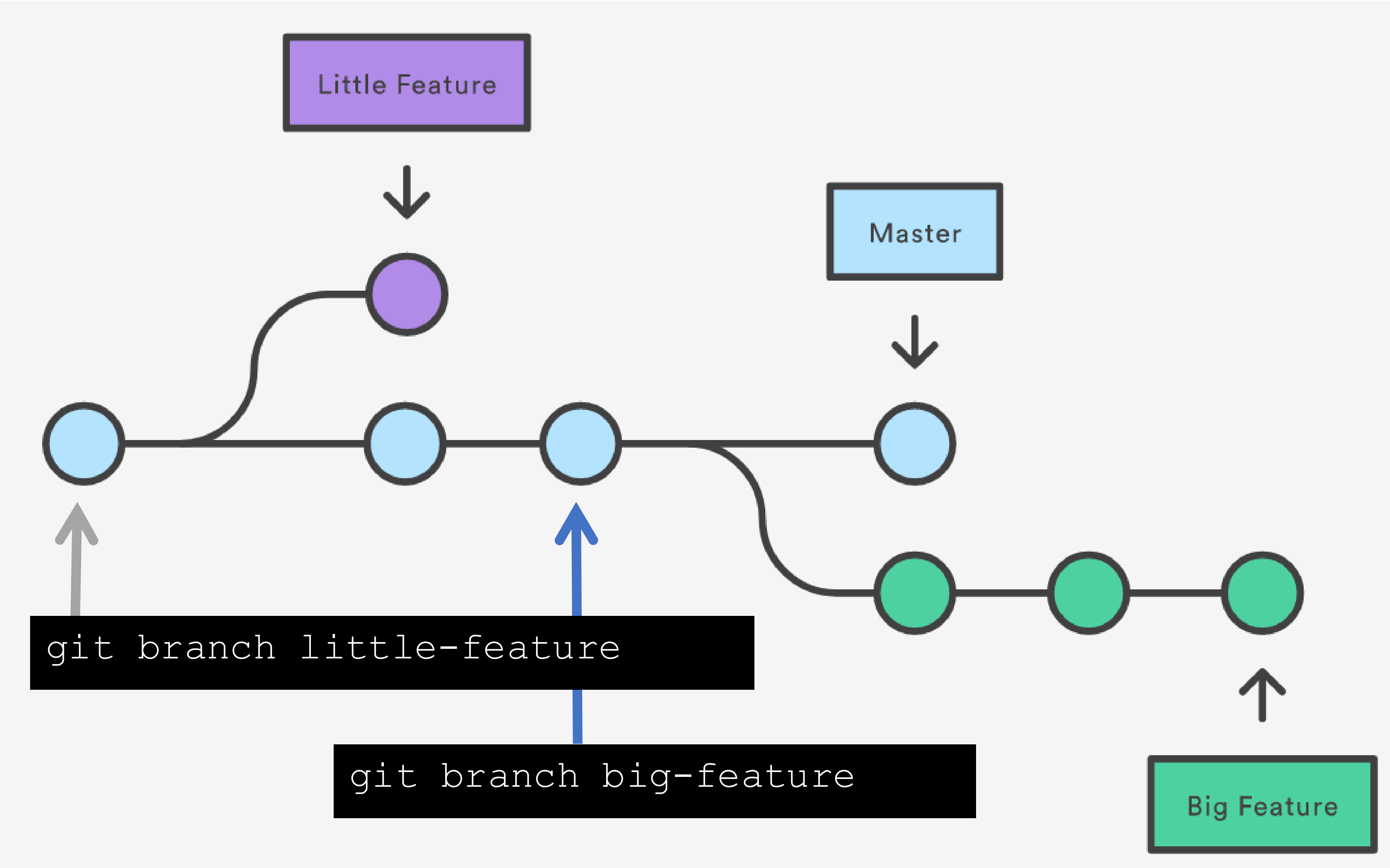

init
clone
repository
staging area
working directory
checkout the project
stage files (add)
commit
status
add
diff
log

Remote Repositories

for backups, collaboration, publishing

- **`remote [nickname] [address]`**
  names remote repository for local use
- **`clone [nickname|address]`**
  makes a local copy of a remote repository
- **`pull –u [remote] [branch-name]`**
  copies changes from remote to local
- **`push –u [remote] [branch-name]`**
  copies changes from local to remote

Branches and Forks


Little Feature
Master
Big Feature

```
git branch little-feature
```

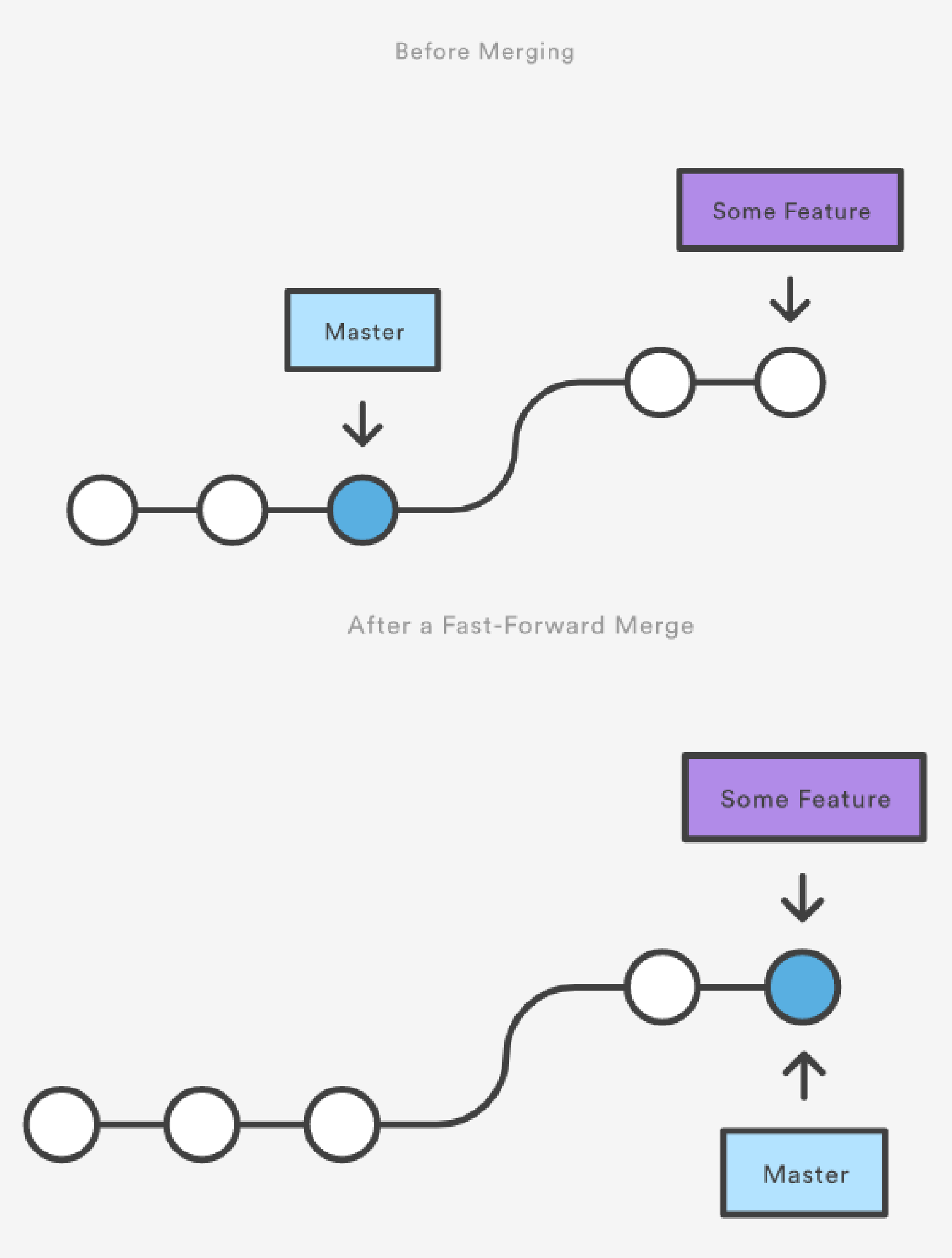
```
git branch big-feature
```

Don’t forget to checkout the branch to use it!

https://www.atlassian.com/git/tutorials/using-branches

Fast-Forward Merge

```
# Create and checkout branch
git checkout -b new-feature master
# Edit some files
git add <file>
git commit -m "Start a feature"
# Edit some files
git add <file>
git commit -m "Finish a feature"
# Merge in the new-feature branch
git checkout master
git merge new-feature
git branch -d new-feature
```

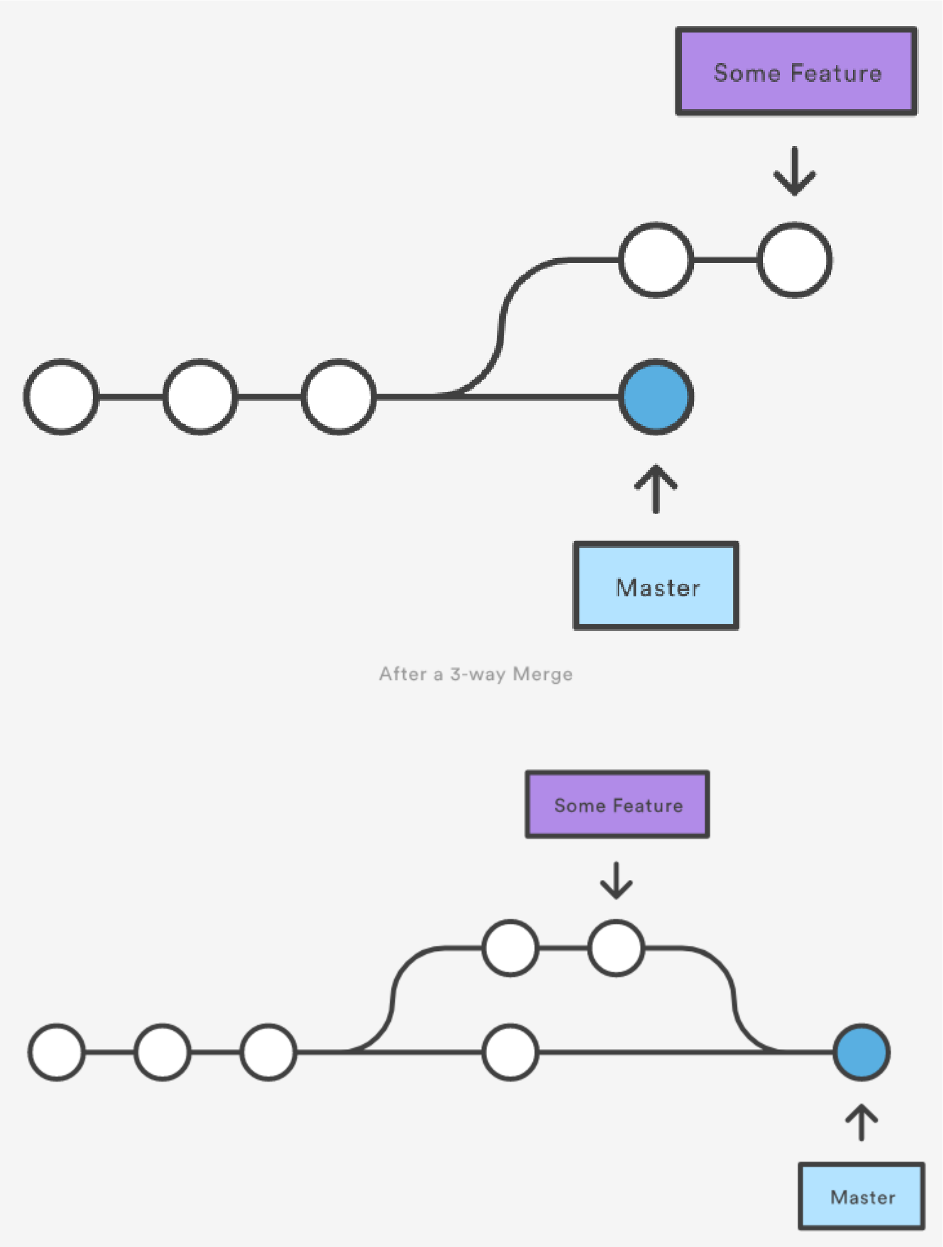

Before Merging
Some Feature
Master
After a Fast-Forward Merge
Some Feature
Master

https://www.atlassian.com/git/tutorials/git-merge

3-way merge

Use **`git status`** and **`git add`** to resolve conflicts

```
# Create and checkout branch
git checkout -b new-feature master
# Edit some files
git add <file>
git commit -m "Start a feature"
# Edit some files
git add <file>
git commit -m "Finish a feature"
# Develop the master branch
git checkout master
# Edit some files
git add <file>
git commit -m
   "Make some super-stable changes to master"
# Merge in the new-feature branch
git merge new-feature
git branch -d new-feature
```


Some Feature
Master
After a 3-way Merge
Some Feature
Master

https://www.atlassian.com/git/tutorials/git-merge

Git resources

Using the CS Gitlab instance, GitHub or BitBucket, do a git tutorial (pick one or two with your team):

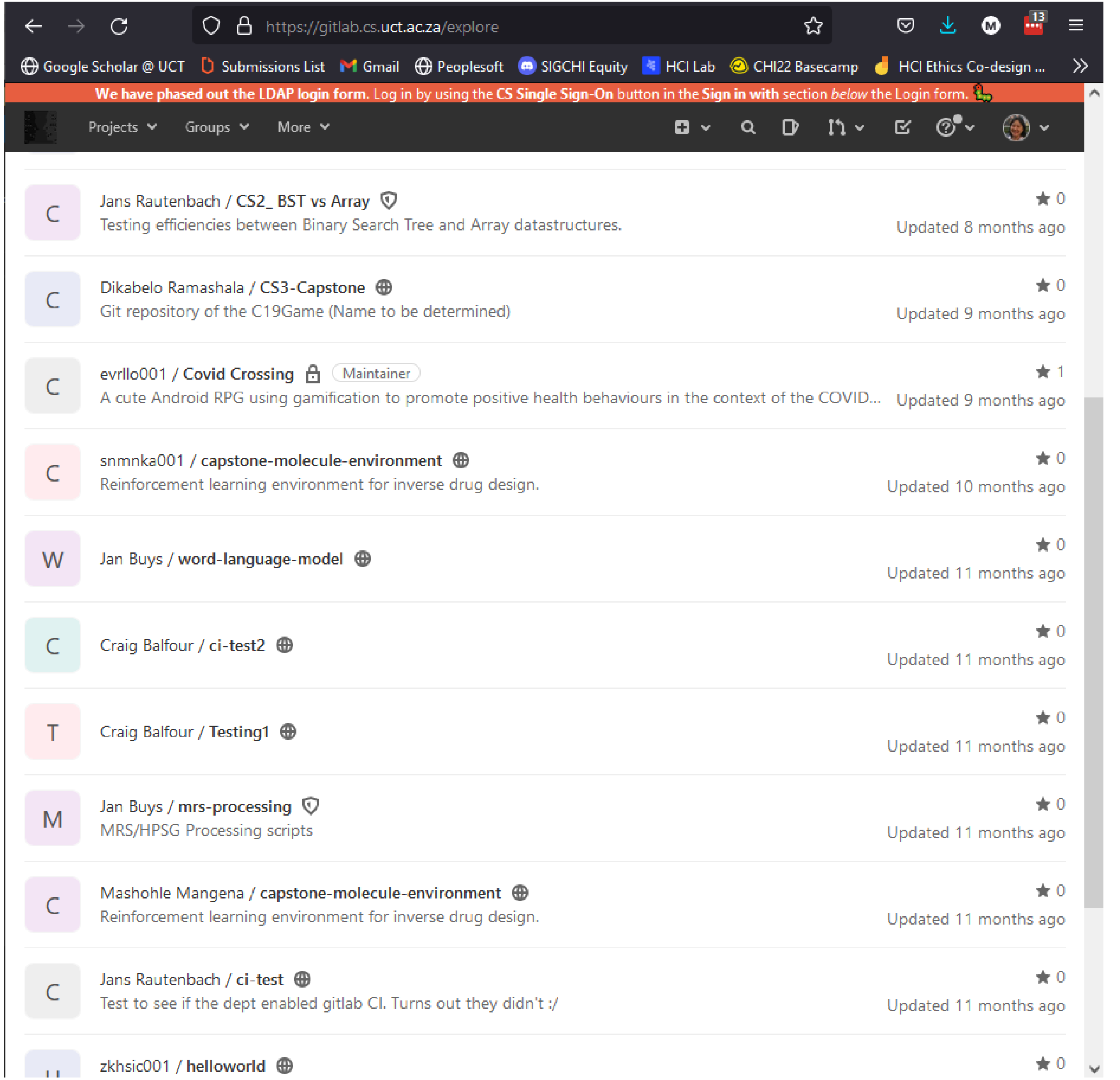
https://www.atlassian.com/git/tutorials

https://try.github.io/levels/1/challenges/1

[LearnCode Academy](https://www.youtube.com/watch?v=0fKg7e37bQE)

https://www.youtube.com/watch?v=0fKg7e37bQE

Linus Torvalds on git
https://www.youtube.com/watch?v=4XpnKHJAok8



We have phased out the LDAP login form. Log in by using the **CS Single Sign-On** button in the **Sign in with** section *below* the Login form.

C	Jans Rautenbach / **CS2_ BST vs Array**	Testing efficiencies between Binary Search Tree and Array datastructures.	★ 0	Updated 8 months ago
C	Dikabelo Ramashala / **CS3-Capstone**	Git repository of the C19Game (Name to be determined)	★ 0	Updated 9 months ago
C	evrllo001 / **Covid Crossing** Maintainer	A cute Android RPG using gamification to promote positive health behaviours in the context of the COVID...	★ 1	Updated 9 months ago
C	snmnka001 / **capstone-molecule-environment**	Reinforcement learning environment for inverse drug design.	★ 0	Updated 10 months ago
W	Jan Buys / **word-language-model**		★ 0	Updated 11 months ago
C	Craig Balfour / **ci-test2**		★ 0	Updated 11 months ago
T	Craig Balfour / **Testing1**		★ 0	Updated 11 months ago
M	Jan Buys / **mrs-processing**	MRS/HPSG Processing scripts	★ 0	Updated 11 months ago
C	Mashohle Mangena / **capstone-molecule-environment**	Reinforcement learning environment for inverse drug design.	★ 0	Updated 11 months ago
C	Jans Rautenbach / **ci-test**	Test to see if the dept enabled gitlab CI. Turns out they didn't :/	★ 0	Updated 11 months ago
	zkhsic001 / **helloworld**		★ 0	